

Functions

DM580: Functional Programming

Casper Bach

Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 4: Defining Functions



Why Study Functions?

Why Functions?

Part I: Mathematics

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch
 - The world is full of dependencies: area depends on radius, position on time

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch
 - The world is full of dependencies: area depends on radius, position on time
 - Computing each case by hand does not scale

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch
 - The world is full of dependencies: area depends on radius, position on time
 - Computing each case by hand does not scale
 - We cannot reason about a pile of instances

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch
 - The world is full of dependencies: area depends on radius, position on time
 - Computing each case by hand does not scale
 - We cannot reason about a pile of instances
- Their cure for the itch: **functions**

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch
 - The world is full of dependencies: area depends on radius, position on time
 - Computing each case by hand does not scale
 - We cannot reason about a pile of instances
- Their cure for the itch: **functions**
 - Calculate output from input. One rule instead of ten thousand cases.

Why Functions?

Part I: Mathematics

- Once upon a time, mathematicians had an itch
 - The world is full of dependencies: area depends on radius, position on time
 - Computing each case by hand does not scale
 - We cannot reason about a pile of instances
- Their cure for the itch: **functions**
 - Calculate output from input. One rule instead of ten thousand cases.
 - Then they spent centuries building tools for reasoning about these things: composition, inversion, calculus.

Why Functions?

Part II: Programming

Why Functions?

Part II: Programming

- Programmers have the same itch: the same loop ten times, one line different

Why Functions?

Part II: Programming

- Programmers have the same itch: the same loop ten times, one line different
 - `sortByTitle(books);`
`sortByAuthor(books);` // same loop, one line different
`sortByYear(books);` // same loop, one line different

Why Functions?

Part II: Programming

- Programmers have the same itch: the same loop ten times, one line different
 - `sortByTitle(books);`
 `sortByAuthor(books);` // same loop, one line different
 `sortByYear(books);` // same loop, one line different
- The cure for the itch is the same:

Why Functions?

Part II: Programming

- Programmers have the same itch: the same loop ten times, one line different
 - `sortByTitle(books);`
`sortByAuthor(books);` // same loop, one line different
`sortByYear(books);` // same loop, one line different
- The cure for the itch is the same:
 - Make the difference an argument: a function

Why Functions?

Part II: Programming

- Programmers have the same itch: the same loop ten times, one line different
 - `sortByTitle(books);`
`sortByAuthor(books);` // same loop, one line different
`sortByYear(books);` // same loop, one line different
- The cure for the itch is the same:
 - Make the difference an argument: a function
 - `sortBy :: (Book → Book → Ordering) → [Book] → [Book]`

Why Functions?

Part II: Programming

- Programmers have the same itch: the same loop ten times, one line different
 - `sortByTitle(books);`
`sortByAuthor(books);` // same loop, one line different
`sortByYear(books);` // same loop, one line different
- The cure for the itch is the same:
 - Make the difference an argument: a function
 - `sortBy :: (Book → Book → Ordering) → [Book] → [Book]`
 - `sortBy byTitle books`
`sortBy byAuthor books`
`sortBy byYear books`

Why Functions?

Part III

Why Functions?

Part III

- You will study functions and functional programming, in Haskell

Why Functions?

Part III

- You will study functions and functional programming, in Haskell
- Many of the ideas are older than programming, and bigger than Haskell

Why Functions?

Part III

- You will study functions and functional programming, in Haskell
- Many of the ideas are older than programming, and bigger than Haskell
- Old ideas from mathematics, applied to the young science of computing

Why Functions?

Part IV

Why Functions?

Part IV

- We are moving towards a world where software is verified correct

Why Functions?

Part IV

- We are moving towards a world where software is verified correct

[HOME](#) [PARTNERS](#) [MOTIVATIONS](#) [RESEARCH](#) [COMPILER](#) [DOWNLOADS](#) [PUBLICATIONS](#)

COMPCERT

COMPILERS YOU CAN *FORMALLY* TRUST



The CompCert project investigates the formal verification of realistic compilers usable for critical embedded software. Such verified compilers come with a mathematical, machine-checked proof that the generated executable code behaves exactly as prescribed by the semantics of the source program. By ruling out the possibility of compiler-introduced bugs, verified compilers strengthen the guarantees that can be obtained by applying formal methods to source programs.

The main result of the project is the CompCert C verified compiler, a high-assurance compiler for almost all of the C language (ISO C 2011), generating efficient code for the ARM, PowerPC, RISC-V and x86 processors.

[Download CompCert C](#) [Read the manual](#)



Why Functions?

Part IV

- We are moving towards a world where software is verified correct

The image shows two overlapping web pages. The background page is the seL4 website, which has a dark blue header with navigation links: HOME, PARTNERS, MOTIVATION, About seL4, Contribute, News & Events, Contact & Support, Become a Member, and Docsite. The main content area features the seL4 logo (a green 'seL4' text) surrounded by a network of green dots and lines. Below the logo are two buttons: 'Learn more about seL4' and 'Start developing'. At the bottom, there is a section titled 'Award-winning technology backed by ground-breaking research' and a paragraph stating 'seL4 is both the world's most highly assured and the world's fastest operating...'. The foreground page is the CompCert project page, which has a white header with the same navigation links. The main content area features the CompCert logo (a yellow laurel wreath with 'ACM Software System Award 2021' text) and a paragraph describing the project: 'The CompCert project investigates the formal verification of critical embedded software. Such verified compilers...'. Below the paragraph are two buttons: 'Download CompCert C' and 'Read the manual'.

seL4

HOME PARTNERS MOTIVATION

About seL4 Contribute News & Events Contact & Support

Become a Member Docsite

The world's most highly assured and fastest operating system kernel

seL4

Learn more about seL4 Start developing

Award-winning technology backed by ground-breaking research

seL4 is both the world's most highly assured and the world's fastest operating

COMPCERT

ACM Software System Award 2021

The CompCert project investigates the formal verification of critical embedded software. Such verified compilers... checked proof that the generated executable code... semantics of the source program. By ruling out the... verified compilers strengthen the guarantees that can... methods to source programs.

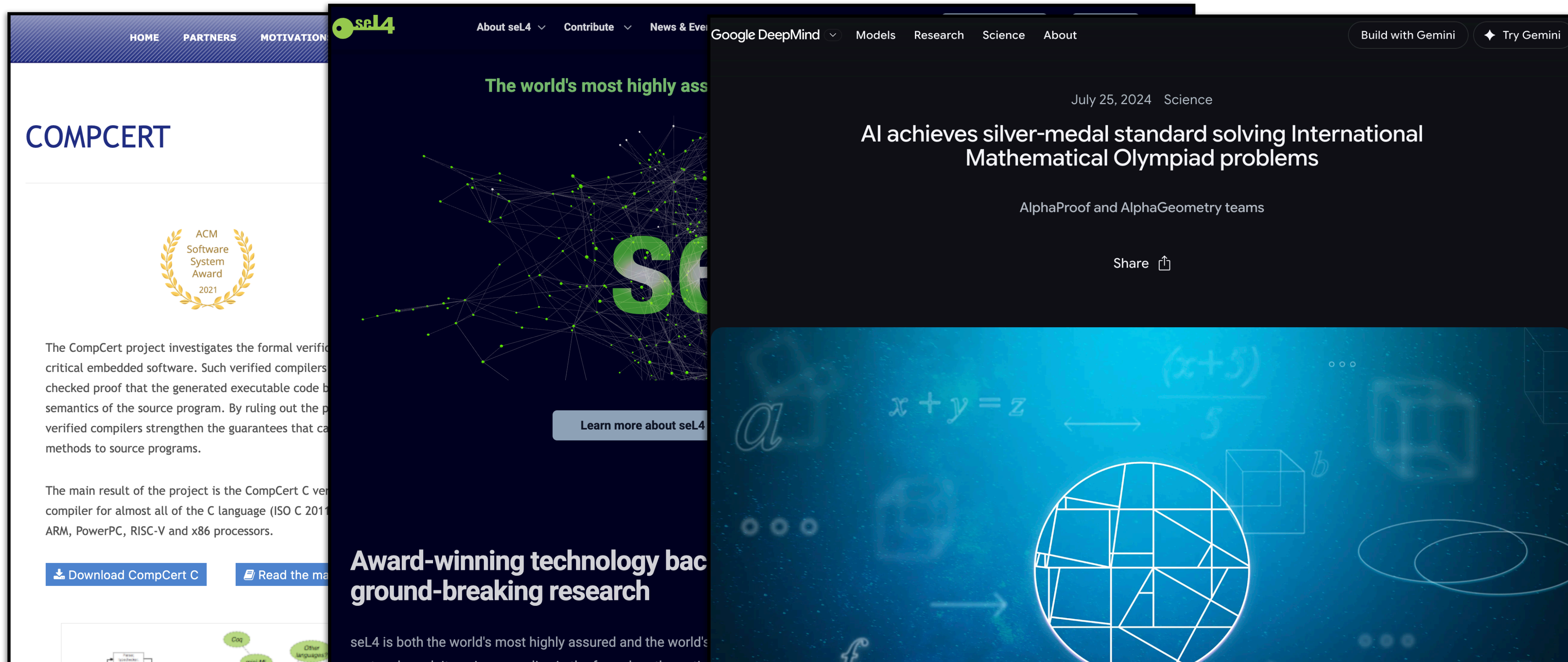
The main result of the project is the CompCert C verified compiler for almost all of the C language (ISO C 2011) on ARM, PowerPC, RISC-V and x86 processors.

Download CompCert C Read the manual

Why Functions?

Part IV

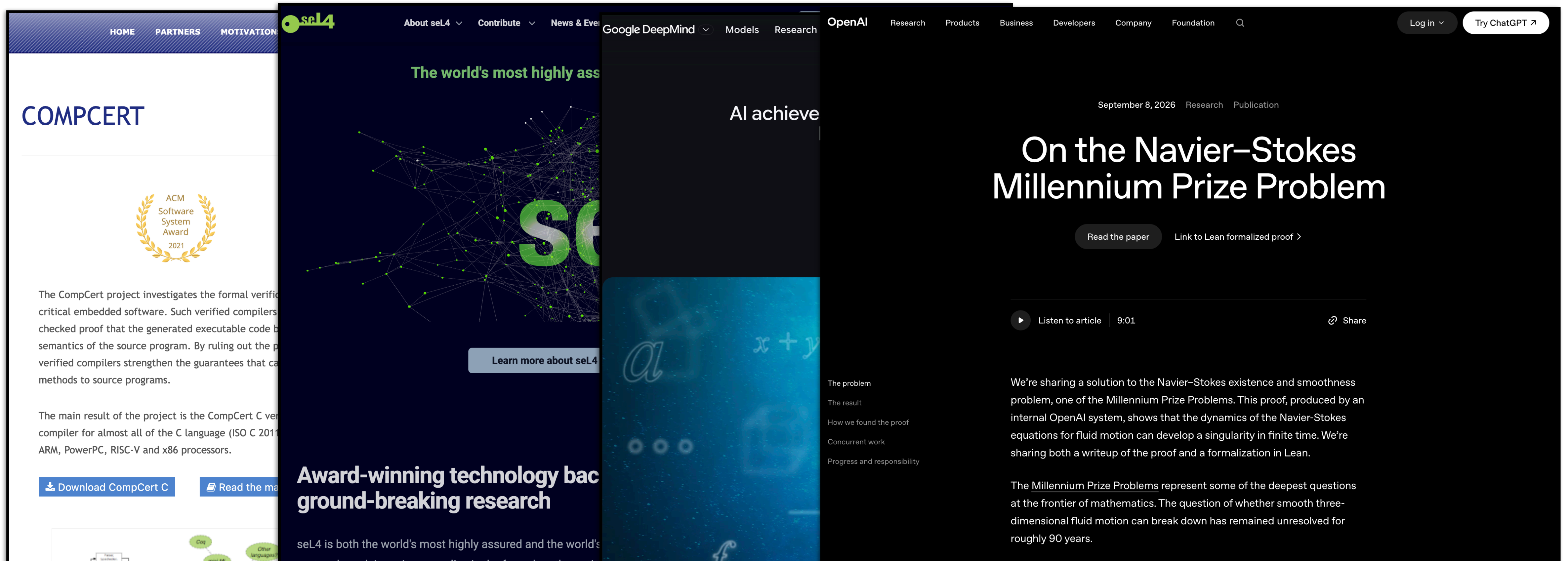
- We are moving towards a world where software is verified correct



Why Functions?

Part IV

- We are moving towards a world where software is verified correct



Why Functions?

Part IV

- We are moving towards a world where software is verified correct

Why Functions?

Part IV

- We are moving towards a world where software is verified correct
- Such verification builds squarely on functional programming

Why Functions?

Part IV

- We are moving towards a world where software is verified correct
- Such verification builds squarely on functional programming
 - The Curry-Howard correspondence:¹ programs $\iff$ proofs

Why Functions?

Part IV

- We are moving towards a world where software is verified correct
- Such verification builds squarely on functional programming
 - The Curry-Howard correspondence:¹ programs $\iff$ proofs
 - Recursive functions $\iff$ inductive proofs

Why Functions?

Part IV

- We are moving towards a world where software is verified correct
- Such verification builds squarely on functional programming
 - The Curry-Howard correspondence:¹ programs $\iff$ proofs
 - Recursive functions $\iff$ inductive proofs
 - Tuples $\iff$ logical conjunction

Why Functions?

Part IV

- We are moving towards a world where software is verified correct
- Such verification builds squarely on functional programming
 - The Curry-Howard correspondence:¹ programs $\iff$ proofs
 - Recursive functions $\iff$ inductive proofs
 - Tuples $\iff$ logical conjunction
 - Data type variants $\iff$ logical disjunction

Why Functions?

Part IV

- We are moving towards a world where software is verified correct
- Such verification builds squarely on functional programming
 - The Curry-Howard correspondence:¹ programs $\iff$ proofs
 - Recursive functions $\iff$ inductive proofs
 - Tuples $\iff$ logical conjunction
 - Data type variants $\iff$ logical disjunction
 - ...

Learning Objectives

- Explain what a higher-order function is
- Explain what higher-order functions are useful for
- Write simple programs that use higher-order functions

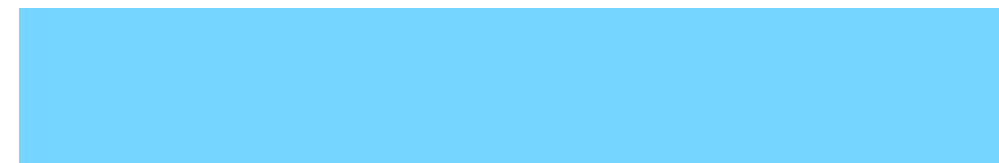
A large, dark gray, stylized lambda symbol (λ) is positioned on the left side of the image, partially overlapping the text.

What is a Higher-Order Function?

Higher-Order Functions

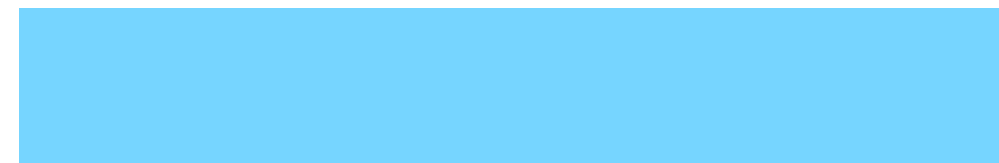
Higher-Order Functions

- A higher-order function is a function that treats other functions as values



Higher-Order Functions

- A higher-order function is a function that treats other functions as values
- Example: map

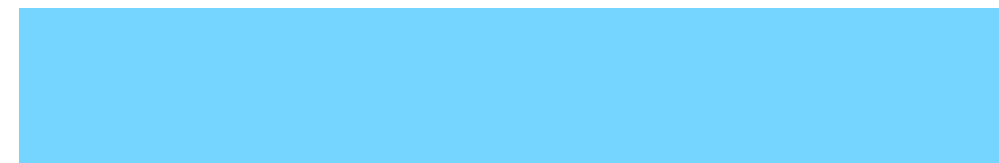


Higher-Order Functions

- A higher-order function is a function that treats other functions as values

- Example: map

`map :: (Int → Int) → [Int] → [Int]`



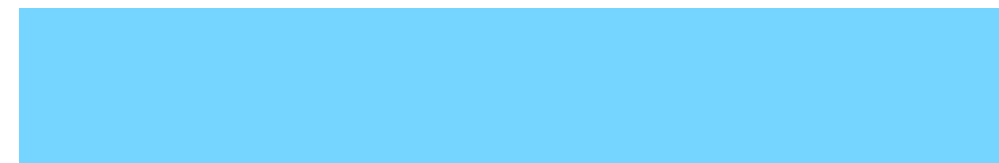
Higher-Order Functions

- A higher-order function is a function that treats other functions as values

- Example: map

`map :: (Int → Int) → [Int] → [Int]`

- Example: λ (written `\` in Haskell)



Higher-Order Functions

- A higher-order function is a function that treats other functions as values

- Example: map

`map :: (Int → Int) → [Int] → [Int]`

- Example: λ (written `\` in Haskell)

`incremented :: [Int] → [Int]`
`incremented xs = map (\x → x + 1) xs`

Example Type: Functions in Haskell

Example Values

- Top-level definitions
 - ▶ $f :: \text{Int} \rightarrow \text{Int}$
 $f\ x = x + 1337$
- Lambdas
 - ▶ $(\backslash x \rightarrow x+1)$
 - ▶ $(\backslash x\ y \rightarrow x+y)$

Operations

- Function application
 - ▶ $f\ 1$
 - ▶ $(\backslash x \rightarrow x+1)\ 3$
 - ▶ $(\backslash x\ y \rightarrow x+y)\ 1\ 2$

Higher-Order Functions

Partial Application

- We've been writing multi-argument functions like this:
 - $\text{add} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
 $\text{add } x \ y = x + y$
- We could equally well write functions like this:
 - $\text{add}' :: (\text{Int}, \text{Int}) \rightarrow \text{Int}$
 $\text{add}' (x, y) = x + y$
- What is the difference?

Higher-Order Functions

Partial Application

Higher-Order Functions

Partial Application

- A *curried* function takes an argument and return a function as result:
 - $\text{add} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
 $\text{add } x \ y = x + y$

Higher-Order Functions

Partial Application

- A *curried* function takes an argument and return a function as result:
 - $\text{add} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
 $\text{add } x \ y = x + y$
- An *uncurried* function takes a tuple of arguments and returns a final result:
 - $\text{add}' :: (\text{Int}, \text{Int}) \rightarrow \text{Int}$
 $\text{add}' (x, y) = x + y$

Higher-Order Functions

Partial Application

- A *curried* function takes an argument and return a function as result:
 - $\text{add} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$
 $\text{add } x \ y = x + y$
- An *uncurried* function takes a tuple of arguments and returns a final result:
 - $\text{add}' :: (\text{Int}, \text{Int}) \rightarrow \text{Int}$
 $\text{add}' (x, y) = x + y$
- Difference: curried functions can be partially applied:
 - $\text{map } (\text{add } 1) [1, 2, 3, 4]$

More Benefits of Programming with Higher-Order Functions



Generic Mapping Over Lists

Generic Mapping Over Lists

- The map function lets us apply a function to all elements of a list

Generic Mapping Over Lists

- The map function lets us apply a function to all elements of a list
- This pattern generalizes to *many* data structures, known as *functors*
 - E.g., we can map over hash-maps, sets, trees, tuples, *sum types*, and many, many more

Generic Mapping Over Lists

- The map function lets us apply a function to all elements of a list
- This pattern generalizes to *many* data structures, known as *functors*
 - E.g., we can map over hash-maps, sets, trees, tuples, *sum types*, and many, many more
- We will consider this generalization in more detail later in the course

Generic Mapping Over Lists

- The map function lets us apply a function to all elements of a list
- This pattern generalizes to *many* data structures, known as *functors*
 - E.g., we can map over hash-maps, sets, trees, tuples, *sum types*, and many, many more
- We will consider this generalization in more detail later in the course
- For now, let us revisit maps vs. for loops

Imperative vs. Functional

*// Imperative:
// Two explicit passes*

```
ArrayList<Double> inEUR
    = new ArrayList<>();
for (int i = 0; i < input.length; i++) {
    inEUR.add(input[i] * 0.15);
}
```

```
ArrayList<Double> finalPrices
    = new ArrayList<>();
for (Double x : inEUR) {
    finalPrices.add(x * 1.20);
}
```

*// Functional:
// Stream pipeline*

```
Arrays.stream(input)
    .mapToDouble(p → p * 0.15)
    .map(p → p * 1.20)
    .boxed()
    .collect(ArrayList::new,
              ArrayList::add,
              ArrayList::addAll);
```

Imperative vs. Functional

```
// Imperative:  
// Two explicit passes
```

```
ArrayList<Double> inEUR  
    = new ArrayList<>();  
for (int i = 0; i < input.length; i++) {  
    inEUR.add(input[i] * 0.15);  
}
```

```
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (Double x : inEUR) {  
    finalPrices.add(x * 1.20);  
}
```

```
// Functional:  
// Stream pipeline
```

```
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

≡ RESULTS ≡

Imperative: 0.44ms avg

Functional: 0.32ms avg

tFun/tImp: 0.72

Imperative vs. Functional

Say we want to refactor the imperative version

```
// Imperative:  
// Two explicit passes
```

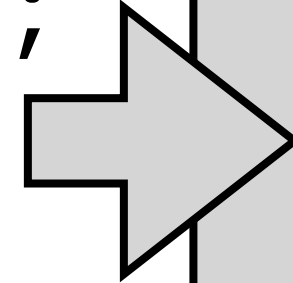
```
ArrayList<Double> inEUR  
    = new ArrayList<>();  
for (int i = 0; i < input.length; i++) {  
    inEUR.add(input[i] * 0.15);  
}
```

```
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (Double x : inEUR) {  
    finalPrices.add(x * 1.20);  
}
```

Imperative vs. Functional

Say we want to refactor the imperative version

```
// Imperative:  
// Two explicit passes  
  
ArrayList<Double> inEUR  
    = new ArrayList<>();  
for (int i = 0; i < input.length  
    inEUR.add(input[i] * 0.15);  
}  
  
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (Double x : inEUR) {  
    finalPrices.add(x * 1.20);  
}
```

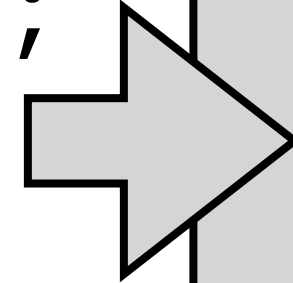


```
// Imperative:  
// One pass  
  
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (int i = 0; i < input.length; i++) {  
    finalPrices.add(input[i] * 0.15 * 1.20);  
}
```

Imperative vs. Functional

Say we want to refactor the imperative version

```
// Imperative:  
// Two explicit passes  
  
ArrayList<Double> inEUR  
    = new ArrayList<>();  
for (int i = 0; i < input.length  
    inEUR.add(input[i] * 0.15);  
}  
  
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (Double x : inEUR) {  
    finalPrices.add(x * 1.20);  
}
```



```
// Imperative:  
// One pass  
  
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (int i = 0; i < input.length; i++) {  
    finalPrices.add(input[i] * 0.15 * 1.20);  
}
```

Is this correct?

Imperative vs. Functional

Say we want to refactor the imperative version

```
// Imperative:  
// Two explicit passes
```

```
ArrayList<Double> inEUR  
    = new ArrayList<>();  
for (int i = 0; i < input.length  
    inEUR.add(input[i] * 0.15);  
}
```

```
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (Double x : inEUR) {  
    finalPrices.add(x * 1.20);  
}
```

Is this correct?

```
// Imperative:  
// One pass
```

```
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (int i = 0; i < input.length; i++) {  
    finalPrices.add(input[i] * 0.15 * 1.20);  
}
```

**Yes: can be shown by loop invariants + glueing
(beyond the scope of this slide)**

Imperative vs. Functional

*// Imperative:
// Two explicit passes*

```
ArrayList<Double> inEUR
    = new ArrayList<>();
for (int i = 0; i < input.length; i++) {
    inEUR.add(input[i] * 0.15);
}
```

```
ArrayList<Double> finalPrices
    = new ArrayList<>();
for (Double x : inEUR) {
    finalPrices.add(x * 1.20);
}
```

*// Functional:
// Stream pipeline*

```
Arrays.stream(input)
    .mapToDouble(p → p * 0.15)
    .map(p → p * 1.20)
    .boxed()
    .collect(ArrayList::new,
              ArrayList::add,
              ArrayList::addAll);
```

Imperative vs. Functional

Say we want to refactor the **functional** version

```
// Imperative:  
// Two explicit passes
```

```
ArrayList<Double> inEUR  
    = new ArrayList<>();  
for (int i = 0; i < input.length; i++) {  
    inEUR.add(input[i] * 0.15);  
}
```

```
ArrayList<Double> finalPrices  
    = new ArrayList<>();  
for (Double x : inEUR) {  
    finalPrices.add(x * 1.20);  
}
```

```
// Functional:  
// Stream pipeline
```

```
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

Imperative vs. Functional

```
// Functional:  
// Stream pipeline  
  
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

Imperative vs. Functional

Say we want to refactor the **functional** version

```
// Functional:  
// Stream pipeline
```

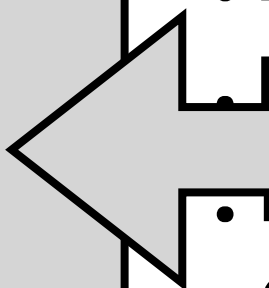
```
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

Imperative vs. Functional

Say we want to refactor the **functional** version

```
// Functional:  
// Stream pipeline  
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15 * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

```
functional:  
Stream pipeline  
  
s.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

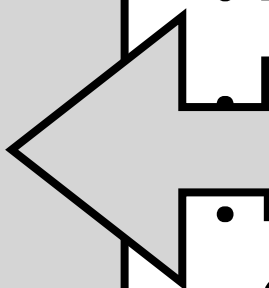


Imperative vs. Functional

Say we want to refactor the **functional** version

```
// Functional:  
// Stream pipeline  
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15 * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

```
functional:  
stream pipeline  
  
s.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```



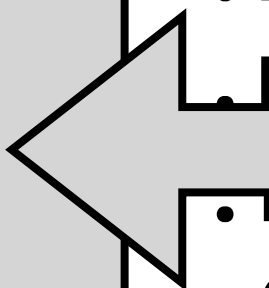
Is this correct?

Imperative vs. Functional

Say we want to refactor the **functional** version

```
// Functional:  
// Stream pipeline  
Arrays.stream(input)  
    .mapToDouble(p → p * 0.15 * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```

```
Functional:  
Stream pipeline  
s.stream(input)  
    .mapToDouble(p → p * 0.15)  
    .map(p → p * 1.20)  
    .boxed()  
    .collect(ArrayList::new,  
              ArrayList::add,  
              ArrayList::addAll);
```



Is this correct?

Yes, by map fusion! Simple equational rewrite.
For any f, g , $\text{map } g \circ \text{map } f = \text{map } (g \circ f)$

Maps vs. For/While Loops

Maps vs. For/While Loops

Point	Loop Version	Map Version
Iteration book-keeping	Manual. Prone to manual indexing/bounds errors.	Absent entirely. Can pass in any function of the right type.

Maps vs. For/While Loops

Point	Loop Version	Map Version
Iteration book-keeping	Manual. Prone to manual indexing/bounds errors.	Absent entirely. Can pass in any function of the right type.
Intermediates	Intermediate data structures not eliminated by compiler. Lists traversed multiple times.	Functions are fused at compile time. List traversed just once.

Maps vs. For/While Loops

Point	Loop Version	Map Version
Iteration book-keeping	Manual. Prone to manual indexing/bounds errors.	Absent entirely. Can pass in any function of the right type.
Intermediates	Intermediate data structures not eliminated by compiler. Lists traversed multiple times.	Functions are fused at compile time. List traversed just once.
Correctness	Per-program loop-invariant reasoning	Fusion law. Proved once. Applicable anywhere.

Maps vs. For/While Loops

Point

Loop Version

Map Version

Imperative code tells us *how*; functional code tells us *what*.

The functional version comes with algebraic laws.

Algebra lets us easily prove programs equal.

Correctness

For program loop invariant
reasoning

Algebraic laws. Proved once.
Applicable anywhere.

Abstracting over Transformations

- There is a whole family of functions that abstract over how we traverse and transform a data structure
- Another very common one, which we will also consider in more detail later is the fold.
- $\text{foldr} :: (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow [b]$

Learning Objectives

- ✓ Explain what a higher-order function is
- ✓ Explain what higher-order functions are useful for
- ✓ Write simple programs that use higher-order functions

A large, dark gray, stylized lambda symbol (λ) is positioned on the left side of the image, extending from the top to the bottom. It has a thick, rounded stroke.

Questions?